

Benjamín Jared Cruzado Fuentes

Documento individual del bloque - Observabilidad

0. Enlace al repositorio y Video

El video demostrativo de la implementación puede visualizarse en el siguiente enlace:

<https://drive.google.com/file/d/18Z0MqzcSrDBxPYnv3bRGFW-iW2IAJaih/view?usp=sharing>

El repositorio del proyecto se encuentra disponible en el siguiente enlace:

Repositorio: <https://github.com/darkkings19/Proyecto-SD-Reserva-de-Hoteles>

La versión correspondiente a esta entrega se encuentra en la rama main.

Dentro de la rama main existe la carpeta Entrega 2 Benjamin Cruzado, la cual contiene los archivos solicitados para esta entrega:

- documento-bloque-observabilidad-entrega2.md (versión en Markdown).
- documento-bloque-observabilidad-entrega2.pdf (versión en PDF).
- link-video-demo-observabilidad.md, que contiene el enlace al video demostrativo.

1. Contexto del sistema

Origen X es una aplicación distribuida para reserva de hoteles. El usuario interactúa desde el frontend, las peticiones entran al API Gateway y luego se comunican con distintos microservicios:

- `user-service`: registro, login, perfil, actualización de usuario, validación de token y sesiones activas.
- `reservation-service`: creación y listado de reservas.
- `inventario-service`: búsqueda de habitaciones y actualización de stock.
- `notification-service`: registro y envío de notificaciones asociadas a reservas.

Mi responsabilidad principal fue el módulo de usuarios y el bloque de observabilidad. Como observabilidad en sistemas distribuidos no se entiende solo mirando un servicio aislado, la implementación terminó cubriendo usuarios y también el flujo distribuido completo de reservas.

2. Descripción del bloque de observabilidad

El bloque de observabilidad permite entender que está ocurriendo dentro del sistema sin tener que entrar directamente al código o a cada contenedor. Para lograrlo se implementaron los tres pilares:

- Métricas: números del sistema, recolectados por Prometheus y visualizados en Grafana.
- Logs: mensajes emitidos por los contenedores, recolectados por Promtail, almacenados en Loki y consultados desde Grafana.
- Trazas: recorrido de una petición entre microservicios, generado con OpenTelemetry, enviado al OpenTelemetry Collector, almacenado en Tempo y visualizado en Grafana.

Con esto se puede responder tres preguntas distintas:

- Prometheus responde: cuánto está ocurriendo y si los servicios están arriba.
- Loki responde: qué mensajes dejaron los servicios al ejecutar una operación.
- Tempo responde: por dónde pasó una petición y cuánto demora cada parte.

3. Flujo observado

El flujo más importante para demostrar observabilidad es la creación de una reserva:

1. El usuario usa el frontend.
2. El frontend llama al API Gateway.
3. El API Gateway recibe la peticion y la envia al servicio de reservas.
4. El servicio de reservas valida el usuario con `user-service`.
5. El servicio de reservas bloquea o actualiza stock con `inventario-service`.
6. El servicio de reservas registra la reserva en su base de datos.
7. El servicio de reservas solicita una notificacion a `notification-service`.
8. Cada servicio emite metricas, logs y spans de trazas.

En Grafana, este flujo se puede ver como datos numericos, mensajes de logs y una traza distribuida de `POST /reservations`.

4. Servicios involucrados

Componente	Responsabilidad en observabilidad
<code>user-service</code>	Expone metricas de usuarios, login, errores, tokens y sesiones activas. Tambien genera trazas mediante el agente de OpenTelemetry para Java.
<code>api-gateway</code>	Es el punto de entrada HTTP. Inicia trazas y propaga contexto hacia las llamadas gRPC.
<code>reservation-service</code>	Expone metricas de reservas, fallos y notificaciones. Propaga trazas hacia usuarios, inventario y notificaciones.
<code>inventario-service</code>	Expone metricas de busquedas, stock y errores. Participa en trazas cuando se buscan habitaciones o se crea una reserva.
<code>notification-service</code>	Expone metricas de notificaciones guardadas, errores y duracion. Participa en la traza cuando una reserva genera notificacion.
Prometheus	Recolecta metricas de los servicios.
Loki	Almacena logs centralizados.
Promtail	Lee logs de contenedores Docker y los envia a Loki.
Tempo	Almacena trazas distribuidas.
OpenTelemetry Collector	Recibe trazas de los servicios y las exporta a Tempo.
Grafana	Permite ver dashboards y explorar metricas, logs y trazas.

5. Decisiones tecnicas

Decision 1: implementar los tres pilares, no solo metricas

Se eligio implementar Prometheus, Loki y Tempo porque el bloque de observabilidad en un sistema distribuido no queda completo solo con metricas. Prometheus permite ver contadores y salud, pero no muestra los mensajes exactos ni el recorrido completo de una peticion.

Alternativa descartada: usar solamente Prometheus y Grafana. Esa opcion era mas simple, pero habria dejado fuera logs centralizados y trazas distribuidas, que eran parte de lo que el profesor esperaba al hablar de observabilidad.

Razonamiento: en una reserva pueden participar cinco servicios. Si algo falla, una metrica puede mostrar que hubo un error, pero no necesariamente explica en que servicio ocurrio ni que mensaje produjo. Por eso se agregaron Loki y Tempo.

Decision 2: usar Grafana como punto visual central

Se eligio Grafana porque permite consultar Prometheus, Loki y Tempo desde una misma herramienta. Esto facilita la demostracion y evita abrir herramientas separadas para cada pilar.

Alternativa descartada: revisar cada herramienta por separado. Prometheus tiene interfaz propia, pero Loki y Tempo se consultan mejor como API o desde Grafana. Usar solo las interfaces separadas hace mas dificil explicar el sistema completo.

Razonamiento: Grafana permite mostrar dashboards para estado general y luego usar Explore para investigar una metrica, un log o una traza especifica.

Decision 3: usar OpenTelemetry Collector antes de Tempo

Los servicios no envian sus trazas directamente como unica estrategia de visualizacion, sino que las mandan al OpenTelemetry Collector. El Collector recibe OTLP y luego exporta a Tempo.

Alternativa descartada: conectar cada servicio directamente a Tempo. Esa opcion reduce un componente, pero hace mas dificil cambiar el destino de las trazas o agregar procesamiento futuro.

Razonamiento: el Collector es una pieza comun en arquitecturas observables porque centraliza recepcion, procesamiento y exportacion de telemetria.

Decision 4: contar usuarios activos con sesiones persistidas

Para mostrar usuarios activos en un instante, no bastaba con contar logins exitosos. Un usuario puede iniciar sesion y despues cerrar sesion o expirar su token.

Alternativa descartada: usar solo JWT stateless sin persistir sesiones. Eso simplifica autenticacion, pero impide saber con certeza cuantos usuarios siguen activos y dificulta revocar sesiones antes de la expiracion.

Razonamiento: se agrego una tabla de sesiones para poder medir `users_active_sessions`, validar tokens y hacer logout real.

6. Metricas implementadas

En usuarios se observan metricas como:

- usuarios creados.
- logins exitosos.
- logins fallidos.
- sesiones activas.
- tokens validados.
- logouts exitosos.
- errores de dominio.

En reservas se observan:

- reservas creadas.
- listados y consultas.
- fallos por etapa.
- notificaciones exitosas o fallidas.
- duracion de creacion de reserva.

En inventario se observan:

- busquedas de habitaciones.
- busquedas sin resultados.
- actualizaciones de stock.
- habitaciones disponibles.
- errores por operacion.

En notificaciones se observan:

- notificaciones guardadas.
- envios externos exitosos o fallidos.
- errores.
- duracion del procesamiento.

7. Dashboards implementados

Se dejaron tres dashboards porque responden a necesidades distintas:

- **User Service Observability**: enfocado en el modulo de usuarios, que fue mi responsabilidad principal.
- **System Observability**: enfocado en metricas del sistema completo, separadas por usuarios, reservas, inventario y notificaciones.
- **Three Pillars Observability**: enfocado en demostrar los tres pilares: metricas, logs y trazas.

El dashboard de tres pilares es el mas importante para defender la decision de observabilidad. El de sistema completo sirve para demostrar que no se observo solo usuarios. El de usuarios sirve para mostrar el modulo individual con mas detalle.

8. Comportamiento ante fallos

Si un servicio cae, Prometheus deja de marcarlo como **UP**. Esto permite detectar rapidamente que el servicio no esta disponible.

Si falla el login por credenciales incorrectas, el modulo de usuarios registra el fallo, incrementa la metrica de logins fallidos y responde con error controlado.

Si se intenta crear un usuario con email repetido, el servicio de usuarios responde con error de dominio e incrementa la metrica asociada.

Si reservas intenta validar un usuario inexistente, **user-service** responde con error y la reserva no se crea. Esto evita generar una reserva asociada a un usuario invalido.

Si inventario no puede bloquear stock, el flujo de reserva falla de forma controlada y se incrementa la metrica de fallos de reserva.

Si notificaciones falla, la reserva ya puede haber sido creada; ese caso se observa con logs y metricas de notificacion fallida. La falla queda visible sin ocultar el problema.

Si Loki o Promtail fallan, el sistema de reservas no se detiene. La aplicacion sigue funcionando, pero se degrada la capacidad de consultar logs centralizados.

Si Tempo u OpenTelemetry Collector fallan, la aplicacion sigue funcionando, pero se pierden o dejan de visualizar trazas distribuidas durante ese periodo.

Si Prometheus falla, los servicios siguen operando, pero se pierde temporalmente la recoleccion historica de metricas.

9. Trade-offs y limitaciones

La observabilidad implementada esta pensada para un entorno academico y local con Docker Compose. En produccion se deberian agregar autenticacion robusta, retencion configurada, backups, alertas y politicas de seguridad.

No se agregaron alertas automaticas. La razon es que la rubrica evaluaba el bloque de observabilidad, no un sistema completo de alertamiento. Aun asi, las metricas necesarias quedan disponibles para construir alertas despues.

Los logs se recolectan desde contenedores Docker con Promtail. Esto es practico para desarrollo, pero en produccion podria requerirse una estrategia mas robusta, logs estructurados y correlacion completa con trace ID.

Las trazas son suficientes para demostrar el flujo distribuido, pero no se configuro muestreo avanzado ni reglas de retencion productivas.

El conteo de usuarios activos requiere persistir sesiones. Esto agrega estado al login, pero se acepto porque permite logout real y una metrica mas precisa de usuarios activos.

Algunas variables y secretos son de desarrollo. En produccion deberian moverse a un gestor de secretos.

10. Como se debe demostrar

Primero se muestra la aplicacion funcionando: frontend, login, busqueda de habitaciones y creacion de reserva.

Luego se muestra Grafana:

1. En el dashboard de tres pilares, se explica que Prometheus muestra metricas, Loki logs y Tempo trazas.
2. En el dashboard de sistema completo, se muestran metricas por servicio.
3. En el dashboard de usuarios, se muestran logins, usuarios creados y sesiones activas.
4. En Explore con Loki, se buscan logs por servicio.
5. En Explore con Tempo, se abre una traza de `POST /reservations` para ver el recorrido entre microservicios.

La idea principal es demostrar que ahora el sistema no solo funciona, sino que tambien se puede observar internamente.

11. Donde ver la observabilidad

La aplicacion y las herramientas quedan disponibles en estas direcciones locales:

Herramienta	URL	Para que sirve
Frontend	<code>http://localhost:3000</code>	Usar la aplicacion, iniciar sesion, buscar hoteles y crear reservas.
API Gateway	<code>http://localhost:8080/health</code>	Verificar que el gateway responda.
Prometheus	<code>http://localhost:9091</code>	Consultar metricas directamente.
Prometheus Targets	<code>http://localhost:9091/targets</code>	Ver si los servicios estan siendo recolectados como UP.
Loki	<code>http://localhost:3100/ready</code>	Verificar que Loki este activo.
Loki Labels	<code>http://localhost:3100/loki/api/v1/labels</code>	Ver etiquetas disponibles para consultar logs.
Loki Services	<code>http://localhost:3100/loki/api/v1/label/service/values</code>	Ver servicios con logs disponibles.
Tempo	<code>http://localhost:3200/ready</code>	Verificar que Tempo este activo.
Tempo Search	<code>http://localhost:3200/api/search</code>	Ver trazas recientes en formato tecnico.
Grafana	<code>http://localhost:3001</code>	Ver dashboards y usar Explore para metricas, logs y trazas.

Grafana usa normalmente estas credenciales:

- Usuario: `admin`
- Password: `admin`

12. Orden recomendado para la demostracion

El orden recomendado para mostrar el trabajo es:

1. Abrir el frontend en `http://localhost:3000`.
2. Iniciar sesion o crear un usuario.
3. Buscar habitaciones y crear una reserva.
4. Abrir Grafana en `http://localhost:3001`.

5. Mostrar `Three Pillars Observability` para explicar los tres pilares.
6. Mostrar `System Observability` para evidenciar que hay metricas de todos los servicios principales.
7. Mostrar `User Service Observability` para explicar el modulo individual de usuarios.
8. Ir a `Explore` con `Loki` y mostrar logs por servicio.
9. Ir a `Explore` con `Tempo` y abrir una traza de `POST /reservations`.